UNIVERSIDAD DA VINCI DE GUATEMALA

FACULTAD DE INGENIERÍA

CARRERA: INGENIERÍA DE SISTEMAS

CURSO: ESTRUCTURA DE DATOS

CATEDRÁTICO: ING. Brandon Chitay

EVALUACIÓN PARCIAL

Elvis Benfel Santay Canto

Carné: 202500113

Guatemala 5 de marzo de 2026

Nombre: ELVIS BENFEL SANTAY CANTO
Carné: 202500113

INTRODUCCIÓN

La notación Big-O es una forma de medir la eficiencia de un algoritmo. Describe cómo crece el tiempo de ejecución o el uso de memoria a medida que aumenta el tamaño de los datos de entrada. Esta notación permite comparar algoritmos y estimar su rendimiento en el peor caso, ignorando constantes y enfocándose en el crecimiento del algoritmo cuando los datos son grandes.

Nombre: ELVIS BENFEL SANTAY CANTO
Carné: 202500113

Descripción de algoritmos

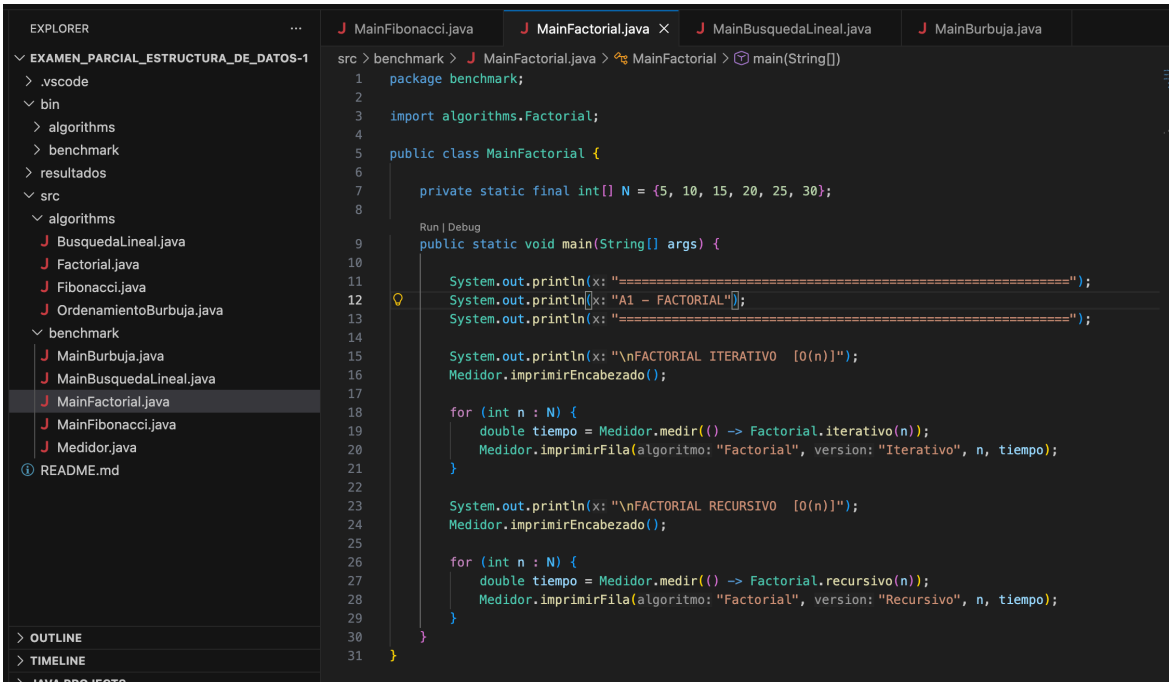
Factorial
El algoritmo factorial calcula el producto de todos los números desde 1 hasta n.
En la versión iterativa se utiliza un ciclo para multiplicar los valores, mientras que en la versión recursiva la función se llama a sí misma hasta llegar al caso base.

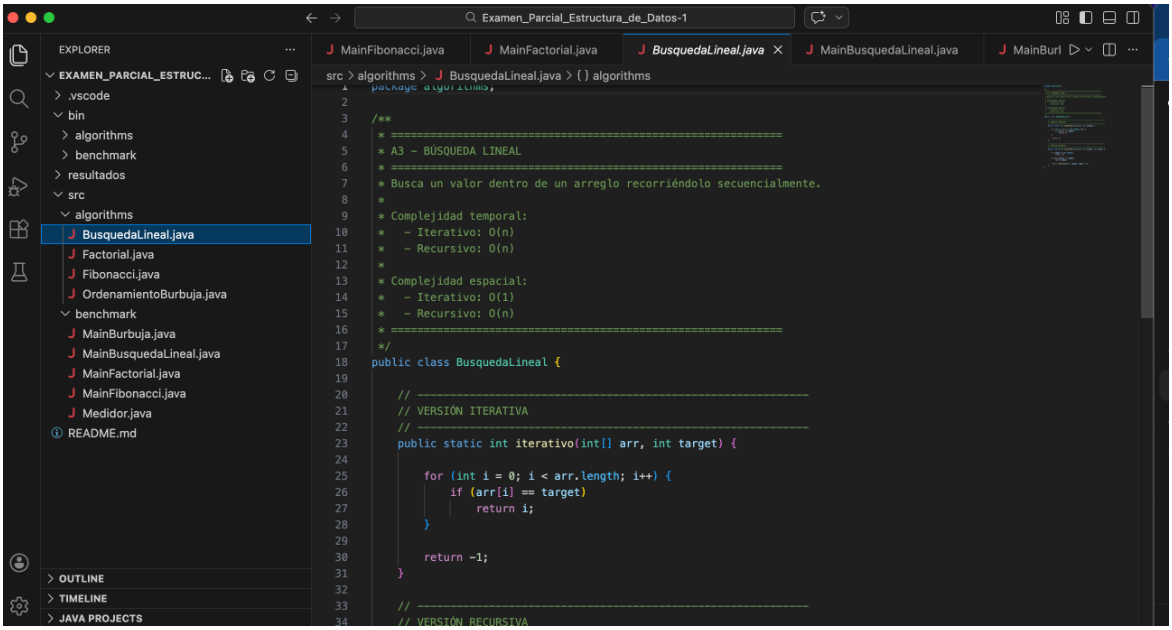
Fibonacci
El algoritmo de Fibonacci genera una secuencia donde cada número es la suma de los dos anteriores.
Puede implementarse con ciclos (iterativo) o con llamadas recursivas que calculan los valores anteriores de la secuencia.

Búsqueda lineal
Este algoritmo busca un elemento recorriendo el arreglo desde el inicio hasta encontrar el valor deseado o llegar al final.
Su complejidad depende del tamaño del arreglo porque puede tener que revisar todos los elementos.

Ordenamiento burbuja
El algoritmo burbuja ordena un arreglo comparando elementos consecutivos e intercambiándolos si están en el orden incorrecto.
Este proceso se repite varias veces hasta que todos los elementos quedan ordenados.

FRAGMENTOS DE CÓDIGO





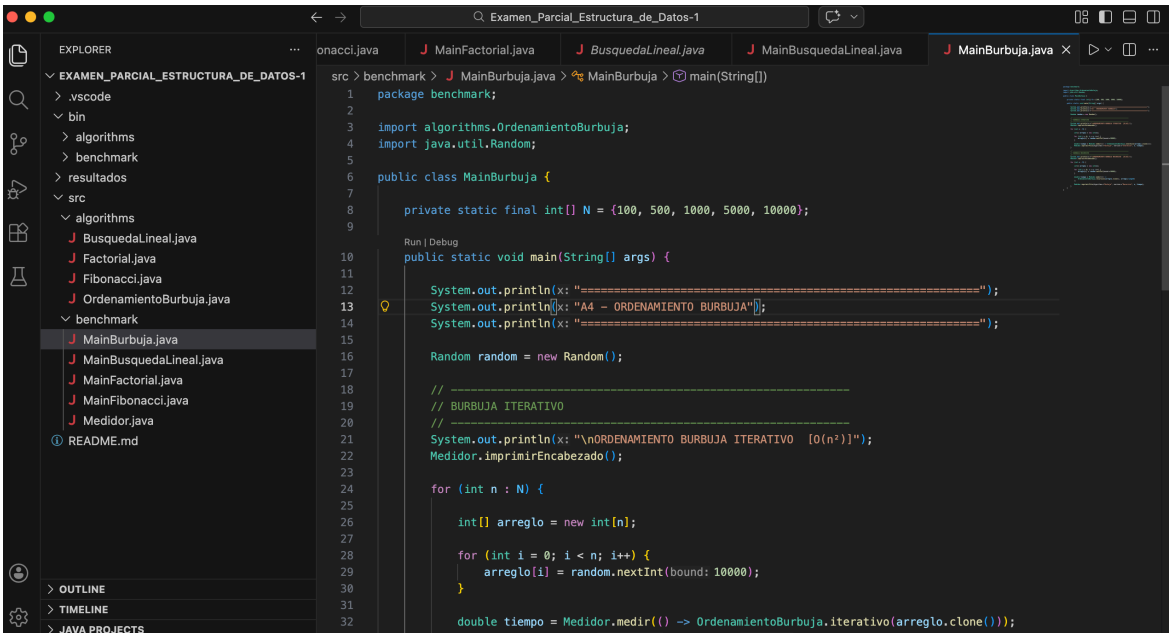

Examen_Parcial_Estructura_de_Datos-1
EXPLORER
onacci.java
MainFactorial.java
BusquedaLineal.java
MainBusquedaLineal.java
MainBurbuja.java
EXAMEN_PARCIAL_ESTRUCTURA_DE_DATOS-1
.vscode
bin
algorithms
benchmark
resultados
src
algorithms
BusquedaLineal.java
Factorial.java
Fibonacci.java
OrdenamientoBurbuja.java
benchmark
MainBurbuja.java
MainBusquedaLineal.java
MainFactorial.java
MainFibonacci.java
Medidor.java
README.md
OUTLINE
TIMELINE
JAVA PROJECTS
src > benchmark > MainBurbuja.java > MainBurbuja > main(String[])
package benchmark;
import algorithms.OrdenamientoBurbuja;
import java.util.Random;
public class MainBurbuja {
private static final int[] N = {100, 500, 1000, 5000, 10000};
Run | Debug
public static void main(String[] args) {
System.out.println(x: "==========");
System.out.println(x: "A4 – ORDENAMIENTO BURBUJA");
System.out.println(x: "==========");
Random random = new Random();
// ----------
// BURBUJA ITERATIVO
// ----------
System.out.println(x: "\nORDENAMIENTO BURBUJA ITERATIVO [O(n²)]");
Medidor.imprimirEncabezado();
for (int n : N) {
int[] arreglo = new int[n];
for (int i = 0; i < n; i++) {
arreglo[i] = random.nextInt(bound: 10000);
}
double tiempo = Medidor.medir(() -> OrdenamientoBurbuja.iterativo(arreglo.clone()));

TABLA DE TIEMPOS MEDIDOS

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

dex_os@Dex-OS Examen_Parcial_Estructura_de_Datos-1 %  /usr/bin/env /Library/Java/JavaVirtualMachines/jdk-17.jdk
/Contents/Home/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/dex_os/Library/Application\ Support/
Code/User/workspaceStorage/9df13988015df1f59230c58d85a99789/redhat.java/jdt_ws/Examen_Parcial_Estructura_de_Dat
os-1_3a8c3501/bin benchmark.MainBusquedaLineal
============================================================
A3 - BÚSQUEDA LINEAL
============================================================

BUSQUEDA LINEAL ITERATIVA  [O(n)]
------------------------------------------------------------
Algoritmo      | Versión      | n        | Tiempo (ms)
------------------------------------------------------------
Busqueda       | Iterativo    | n=100    | 0.032679 ms
Busqueda       | Iterativo    | n=500    | 0.006538 ms
Busqueda       | Iterativo    | n=1000   | 0.012775 ms
Busqueda       | Iterativo    | n=5000   | 0.022957 ms
Busqueda       | Iterativo    | n=10000  | 0.010801 ms

BUSQUEDA LINEAL RECURSIVA  [O(n)]
------------------------------------------------------------
Algoritmo      | Versión      | n        | Tiempo (ms)
------------------------------------------------------------
Busqueda       | Recursivo    | n=100    | 0.010400 ms
Busqueda       | Recursivo    | n=500    | 0.003123 ms
Busqueda       | Recursivo    | n=1000   | 0.005299 ms
Busqueda       | Recursivo    | n=5000   | 0.029650 ms
Busqueda       | Recursivo    | n=10000  | 0.011744 ms
dex_os@Dex-OS Examen_Parcial_Estructura_de_Datos-1 %
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

dex_os@Dex-OS Examen_Parcial_Estructura_de_Datos-1 %  cd /Users/dex_os/Desktop/Examen_Parcial-Estructura_de_Dat
os/Examen_Parcial_Estructura_de_Datos-1 ; /usr/bin/env /Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/Ho
me/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/dex_os/Library/Application\ Support/Code/User/wo
rkspaceStorage/9df13988015df1f59230c58d85a99789/redhat.java/jdt_ws/Examen_Parcial_Estructura_de_Datos-1_3a8c350
1/bin benchmark.MainFactorial
============================================================
A1 - FACTORIAL
============================================================

FACTORIAL ITERATIVO  [O(n)]
------------------------------------------------------------
Algoritmo      | Versión      | n        | Tiempo (ms)
------------------------------------------------------------
Factorial      | Iterativo    | n=5      | 0.067846 ms
Factorial      | Iterativo    | n=10     | 0.012605 ms
Factorial      | Iterativo    | n=15     | 0.025848 ms
Factorial      | Iterativo    | n=20     | 0.025047 ms
Factorial      | Iterativo    | n=25     | 0.018653 ms
Factorial      | Iterativo    | n=30     | 0.031796 ms

FACTORIAL RECURSIVO  [O(n)]
------------------------------------------------------------
Algoritmo      | Versión      | n        | Tiempo (ms)
------------------------------------------------------------
Factorial      | Recursivo    | n=5      | 0.004615 ms
Factorial      | Recursivo    | n=10     | 0.003138 ms
Factorial      | Recursivo    | n=15     | 0.004951 ms
Factorial      | Recursivo    | n=20     | 0.006548 ms
Factorial      | Recursivo    | n=25     | 0.010147 ms
Factorial      | Recursivo    | n=30     | 0.007807 ms
dex_os@Dex-OS Examen_Parcial_Estructura_de_Datos-1 %
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

dex_os@Dex-OS Examen_Parcial_Estructura_de_Datos-1 %  /usr/bin/env /Library/Java/JavaVirtualMachines/jdk-17.jdk
/Contents/Home/bi
n/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/dex_os/Library/Application\ Support/Code/User/workspa
ceStorage/9df1398
8015df1f59230c58d85a99789/redhat.java/jdt_ws/Examen_Parcial_Estructura_de_Datos-1_3a8c3501/bin benchmark.MainBu
rbuja
============================================================
ORDENAMIENTO BURBUJA
============================================================

ORDENAMIENTO BURBUJA ITERATIVO  [O(n²)]
------------------------------------------------------------
Algoritmo      | Versión      | n        | Tiempo (ms)
------------------------------------------------------------
Burbuja        | Iterativo    | n=100    | 0.172513 ms
Burbuja        | Iterativo    | n=500    | 0.872230 ms
Burbuja        | Iterativo    | n=1000   | 1.170807 ms
Burbuja        | Iterativo    | n=5000   | 20.418943 ms
Burbuja        | Iterativo    | n=10000  | 106.875002 ms

ORDENAMIENTO BURBUJA RECURSIVO  [O(n²)]
------------------------------------------------------------
Algoritmo      | Versión      | n        | Tiempo (ms)
------------------------------------------------------------
Burbuja        | Recursivo    | n=100    | 0.101229 ms
Burbuja        | Recursivo    | n=500    | 0.531213 ms
Burbuja        | Recursivo    | n=1000   | 0.727542 ms
Burbuja        | Recursivo    | n=5000   | 18.469183 ms
Burbuja        | Recursivo    | n=10000  | 91.386397 ms
dex_os@Dex-OS Examen_Parcial_Estructura_de_Datos-1 %  cd /Users/dex_os/Desktop/Examen_Parcial-Estructura_de_Dat
```

LINK DEL REPOSITORIO:

https://github.com/benfel2993-source/ED_Parcial_ElvisSantay

Análisis y conclusiones

Los resultados muestran que los algoritmos **iterativos suelen ser más eficientes** que los recursivos porque evitan el costo adicional de las llamadas a funciones.
Los algoritmos de **búsqueda lineal tienen complejidad O(n)** porque revisan cada elemento una vez.
El **ordenamiento burbuja tiene complejidad O(n²)** debido a sus ciclos anidados.
En general, la notación Big-O permite entender cómo aumenta el tiempo de ejecución cuando el tamaño de los datos crece.

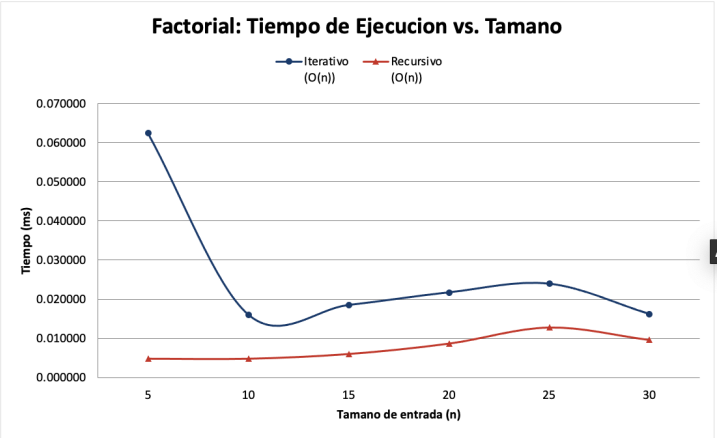
GRÁFICAS

HOJA — A1 - Factorial | Iterativo vs. Recursivo

Big-O Iterativo:	O(n)	Big-O Recursivo: O(n)

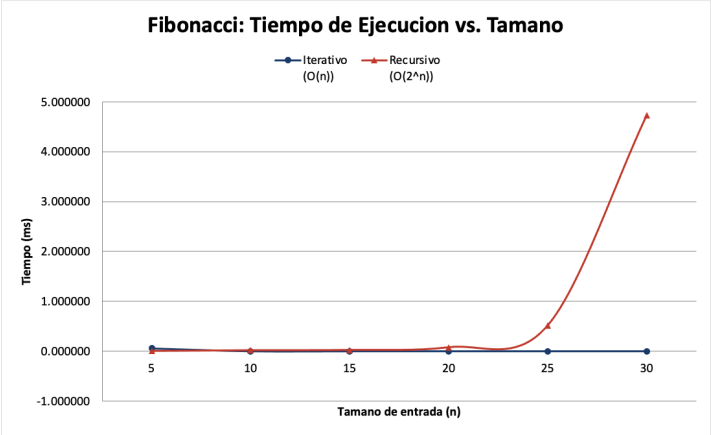
n	Iterativo (O(n))	Recursivo (O(n))
5	0.062347	0.004829
10	0.016009	0.004828
15	0.018480	0.006037
20	0.021714	0.008708
25	0.023923	0.012849
30	0.016167	0.009639

Observacion: Ambas versiones son O(n): el tiempo crece linealmente con n.



HOJA — A2 - Fibonacci | Iterativo vs. Recursivo

Big-O Iterativo:	O(n)	Big-O Recursivo: O(2^n)

n	Iterativo (O(n))	Recursivo (O(2^n))
5	0.058533	0.002163
10	0.000630	0.016634
15	0.000571	0.022536
20	0.000630	0.076810
25	0.000910	0.512066
30	0.000831	4.729404

Observacion: Recursivo crece exponencialmente: duplicar n casi cuadriplica el tiempo.